

Assignment 3

INTE2402/2691 Cloud Security

Q1. Data Privacy Protection with Homomorphic Encryption

(1) Use the tool https://www.mobilefish.com/services/rsa_key_generation/rsa_key_generation.php to generate your public key for Paillier encryption and determine your private key for Paillier decryption. The size of the modulo is required to be 1024 bits.

Large prime numbers generated using the tool in decimal (I get it in base-10 decimal because then it makes calculations easy);

p =

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466183

q =

7265014247994514177821069189687475188235420333174159337772714488542014180095534
459605814223234694007643695864714082176479666655133115079533744379041150729

Input online RSA key generation:

Step 1: Enter or generate prime numbers

Generate prime numbers (p,q).
The key size is*: bits Auto generate prime number p and q

-- Or --

Prime number (p) is a *: Bitsize: 512

120936074897807123612131014690932358351511066746831
305890807771302570556181724741253948173448929311395
232205918240777152075941308259631771259487943714661
83

Enter prime number (p)*:

Prime number (q) is a *: Bitsize: 512

726501424799451417782106918968747518823542033317415
933777271448854201418009553445960581422323469400764
369586471408217647966665513311507953374437904115072
9

Enter prime number (q)*:

Paillier private key is 2 prime numbers: (p, q) (This is represented as a tuple)

Private key =

(1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466183,
7265014247994514177821069189687475188235420333174159337772714488542014180095534
459605814223234694007643695864714082176479666655133115079533744379041150729)

Paillier public key;

1. Calculate the Modulus (N) by multiplying two prime numbers together.

$N = p \times q$

N =

8786023072291004631961218194243719321460228445547562509128360768199295921522649
0856758224550513278708321663845655307019646460432143539321675694124156348840682
0021280292850753185973277358692034023375812200468934375999358404963968241804684
97614803478192099344919454913905244029506642199359287869892287229297407

2. Calculate the Generator (g) by adding 1 to your modulus.

$g = N + 1$

g =

8786023072291004631961218194243719321460228445547562509128360768199295921522649
0856758224550513278708321663845655307019646460432143539321675694124156348840682
0021280292850753185973277358692034023375812200468934375999358404963968241804684
97614803478192099344919454913905244029506642199359287869892287229297408

```
= RESTART: C:/Users/IshanKumarasinghe/Downloads/Assignment_3_kamesh/ql_homomorph  
ic_encryption.py  
N: 8786023072291004631961218194243719321460228445547562509128360768199295921522  
64908567582245505132787083216638456553070196464604321435393216756941241563488406  
82002128029285075318597327735869203402337581220046893437599935840496396824180468  
497614803478192099344919454913905244029506642199359287869892287229297407  
g: 8786023072291004631961218194243719321460228445547562509128360768199295921522  
64908567582245505132787083216638456553070196464604321435393216756941241563488406  
82002128029285075318597327735869203402337581220046893437599935840496396824180468  
497614803478192099344919454913905244029506642199359287869892287229297408
```

3. Then public key is: (N, g) (This is represented as a tuple)

Public Key =

(8786023072291004631961218194243719321460228445547562509128360768199295921522649
0856758224550513278708321663845655307019646460432143539321675694124156348840682
0021280292850753185973277358692034023375812200468934375999358404963968241804684
97614803478192099344919454913905244029506642199359287869892287229297407,
8786023072291004631961218194243719321460228445547562509128360768199295921522649
0856758224550513278708321663845655307019646460432143539321675694124156348840682
0021280292850753185973277358692034023375812200468934375999358404963968241804684
97614803478192099344919454913905244029506642199359287869892287229297408)

(2) Use your public key to encrypt your monthly incomes from January to December in 2025 and upload the ciphertexts to the cloud. Assume that your monthly income is MD5(your student ID||the month) (mod 10000). What are the twelve ciphertexts (r should be chosen randomly)?

$$\text{Income} = \text{MD5}(\text{StudentID} || \text{Month}) \pmod{10000}$$

I will show the calculation steps for first month of 2025 income,

StudentID = S4129857

Input StudentID and Month to MD5 algorithm and get the output hexadecimal value,

MD5 (Hex) = a4ef871b9a3c9fc3bb6d1b8326970721

Convert the hexadecimal value to decimal value,

MD5 (Decimal) = 219237090570106889348642864501976663841

Then do the modulo operation by 10000, then we get plaintext income (m)

Month 1: Income = 3841

Encryption steps,

The Paillier encryption formula is $c = g^m \cdot r^N \pmod{N^2}$

1. Choose a random integer r_1 such that $0 < r_1 < N$ and $\text{gcd}(r_1, N) = 1$

Random number =

8663686229926538813224463523546300593446854852345479467887222073548100228863768
7043216595721416170552950861205150432446981517605751498417779446091147338066318
1711972560481589354581077890427766395835320067796345441350989782942510398596797
09617102035604016117961991993466182747101462826677001229345537139973314

2. Compute $c_1 = g^{8882} \cdot r_1^N \pmod{N^2}$

Cipher text =

7008137910189296061662355102060788104815565317459021178589599674213174014083412
9504035598088970607312582536802511497949419305776944582052957176003153088754011
7429607931869263311676281811468191017867845068971466172602716675771850516688229
6563527954878626497998245977516505203285663289036339695246798445272839235184417
1866513231243520732073050070809665745129957877997601333386209188607002605976526
2767309278507271805329432375302856510705159584091767114470607366430305010779404
3158342718588756738801759719195125573667406764161748313030565062848013698676636
397452732766131013106059114915646285457680735848247886134616253

Like this calculate and get the Ciphertext for each month,

S41298571
MD5 Hash value: a4ef871b9a3c9fc3bb6d1b8326970721
Decimal value: 219237090570106889348642864501976663841
Random Number: 866368622992653881322446352354630059344685485234547946788722207354810022886376870432165957214161705529508612051504324469815176055715498417779446091
147330663181711972560481589935450107789042776639583532006779634544138098978294251039859679709617102035604016117961991993466182747101462826677001229345537139973314
Cipher text: 7008137910189296061662351026078810481556531745902117858959674213174014083412950403559808897060731258253680251149794941930577694458205295717600315
3088754011742960793186926331167628181468191017867845068971466172602716675717850516682296563527954878626497998245977516505203285663289036339695246798445272839235
184417186651323124352073207305007080966574512995787797967013338620918860700260597652627673082718053294323735028565107051595840917671144706073664303050107794
043158342718588756738801759719195125573667406764167174931303056506284801369867663697452732766131013106059114915646285457680735848247866134616253
Month 1: Income = 3841
S41298572
MD5 Hash value: c75e5b52a3a30a6c3a3c8a48f0fda47
Decimal value: 265006299312526951706209727718146169415
Random Number: 67667157099585905855377551421640693770095427573860028833913774018214520514191803697238274757255890614806812080037331597767976392120207597431838
30027906586389129564375182503872763537370896442615029871851383072289595661987247750180070458556977784864879206353502394932159220396340808501805221954825643156008
Cipher text: 65246102107071445075834543242888995618455770302575843238244222030868390569837724388232484657210015546580378665321560606732952623607135127631733594
96804347274882932737504686637211564947066929008674135884698497575213266520946704785272689239312657689326247450341450643717193143479892709457821132869820170369420
028544229560131915944208319228280197967816010163046636872851469051335497378183033792206407457757533110214180992674329938494749739826822042092985823149999782165050
2046346769460164025310529058710949641509146308489085441215156844796738424616188030242891439688337215314913633592790204565503275230527943703049
Month 2: Income = 9415
S41298573
MD5 Hash value: 7a9belb6cf206c7d04cd0ca0577c93d83
Decimal value: 162975199524621628979657228875140251011
Random Number: 87829230016665265195771779254875409820046278448828108426616078474220065717401086214038900169126331449594019956413272720604990093488675265722968296
752705066944197860399718886226591382849938165918554321132101997935326060051624663156337676534094528535680582004038729808611110933157160248332928679545821629616022
Cipher text: 1496636656910247253544603369461840870368511172561548646096989757396940955244910705441306811491341490036511839627972474495709525329130748682875515757
961966701538575068205024683226764061471821009648755155508612673174315163846415420940453516242960335632489014350929806331642028246987130915660675860227835452190
07079664689954540453411249702527374814119973900402875107045891622167796198131334090251338156047870697365100121395205625230395009320456305559810711313101415864989
27856069708087638474176800037393267303641089550632048842230194811535781981273482438644197162741092266353982010253077642039494751280809276869476
Month 3: Income = 1011
S41298574
MD5 Hash value: cc8c0cf08304cb0685f62e006e5f0d74
Decimal value: 271889695144540291934461122531427280494
Random Number: 3158232049336981367879894705249947410696046213351534760424705442104711210942693079031755450555304394049143709864500248683882201843472595958621093
1230467843832904795047466121572698108613350911360068958376151524011195118269366065903516761360348033689256155435831233439416968633014235735425738615109119601302440
Cipher text: 44101036499181872776892040171036714727553759527451479988634967734453027058274621463020033183707586500612615892600558473137797994644733502074
7711797242003796439991101081580844460231462498137618447192031466816881598471540141650972348188479860743824754565599593267052978909130505416528678481777469197531
612350961813022835522801362322066356828503278225663752479140283506829059606440506644987904447174622279713081146596243433595959442343537265871369552758434819
351685921025890668687662079091373065726706408963255637515912189635572120217711789337347039896407989722944939168032378822817881755206448268419
Month 4: Income = 484

S41298575
MD5 Hash value: 2c792838fa5ff5c1ad0ed7975a10c2c16
Decimal value: 59115116074479589702664221576101899286
Random Number: 747312648122482438222274970422361009677553421425512778798172881970710620426451975301877823279395123123242601616993326075290533576244464470641380
55710889590516584842687580362136969533155254267369728934399232778736019830815003079786086879045466082734494487087126346507502680706518455206500529762799052499
Cipher text: 87234222122893066239237404672382795999665949255099086935731372034387442031400501815683637865031458159751373792992908365171990340071722991725141765
1984255350247792805482290240559248206010965240545462257078140893461839216857758598078447364927542738660530859765442819981085485520140716525316630100891502138280
805916439445895520240174734788141845910486783453043531845543237356589215905924387713962305659376916999258324729217200732329942557073380670676019442931099064733848
3918163577589346225713777375251047639125901754688711040091562221225876789046755469320434721561248120572180432687233639242655590750454505993379
Month 5: Income = 9286
S41298576
MD5 Hash value: acb9bb40097144c5f48590e5dd96ef855
Decimal value: 22959158807795505550457844772033677397
Random Number: 53081607947197640521048521987339089870382196358368993305001501609015878258220528100622634135368836828726154223900570590966471431876141628817615687
584037644203931714555269267151177657545315409974780260024131810167142560318492519574816041336574792168070417988772642736739377545469156047709006967916271691962518
Cipher text: 1594006397325159679345491585398082174552504661982474906824367208177359884014806267181115820938657601229944285391589504334722256139240029158394939711
079042134298151385327852551791904803192761311480795761380697649098561709693444746589440052406272079908337248868307978827636405993460156611604287863980671490096941
455416911307212343240752347379086928283170289616262471299860992418740148893118832839323555119132735367156753618154625456284118173100770002896160409824669896221793
440868967264048185860614934001372906734939881976686093405664059173877734948237890808343367394408572391638674912925909131290679144957775072445642
Month 6: Income = 7397
S41298577
MD5 Hash value: ee9e955283619f84f22933dd7182cf5
Decimal value: 3171796745168593061847721010480406387957
Random Number: 82562208903623588182300104087378685535504431005277736077904935352380204884943082267852472118487897674417800158046823060384269671896832636991228084
8219097271343658525988025187769370008104711534831556605862062891190580157307961664093015188509030307573087491817938542796502753412656444175926195567705235957533
Cipher text: 5561642026882807164003932794536494865962195206600755273630078319397722776056145642176166815206412127238160911702323159737037336375845994279911039712
2381507552310528382937713057535947700923351273858910811727793619050673463146172247157674619113062850480889129834662865947241039747960785907876870204659033352113
2140713872657813685850872956706981406978796569413406540791292798071578020134810022273182490382560563043156754154703374588862936535698812355278096773688469093918
48427762183164422128306142119363540730312910605571210989818579143780158478462081690872138741516793682024728245266530703053173680545548381353223
Month 7: Income = 7957
S41298578
MD5 Hash value: b54f2076cd68147b7ded5375475c2e5f
Decimal value: 241001117138494769938857229040301649503
Random Number: 87515833975826385317144739580811440744524314211594085104014871639015515593383133575570791193528478326834938057453849971520541690124912810848361911
73613301950294516747710732302777060454845431370064785426158084111371102106078621028429031251988891126535281750613363143091587378737243925685379347368991659682
Cipher text: 42945844403284986764799973677963925418866407293273530615896208946619273970592684183852499231690564523958037089959336435111675365786580879994463502767
24593551613995989307379033872267603011697665291681671354967374439694297459335974475721282387915941978399628452921608816053839617533194691830216380291350765908149
310802797197195315478777627955477023112304611919486696203884762359073788232784159748590046834389364324185673862658649945927513089987715634779067504484417803737446
3466703235162823672674772788565978417136893279088308592091506398434933159383795076917713417679193662957352152350880808378725490441320718390759440
Month 8: Income = 9503

```

S41298579
MD5 Hash value: 19b4427f6a8a56a19be0eb6d5a2887fd
Decimal value: 3416662063142298026394475838155819005
Random Number: 97159722429307793337062952770148576911030442250797377392517802369721536204425192589702272025811081299917419606178706678693653851135557504052068341
41704703539240993517776197513205149526220265592941085597516488683313841843126997768595258425263269988077702901059029947035418162581382584047784728885320377714
Cipher text: 23612347441539588365918170946229214626090815762334657070998853935743282344040530105626217830429329861773431093333693269991444104621680182909853993
780207988812798549731023360733046500650013674511960570553656565637946753734867599430916522989159793619646923650439410502487964283392755267930043302388952699225198
151344531495663479402701322146729378543945569204940625327044766690386002696949406957867579323435136537525905238014409039674387695496660394945768039444998373326409
25537307846570757721469658104730349621394936206347143901342188882091930519473999692422704863351220534050932734095989002596753739258978314575827
Month 9: Income = 9005
S412985710
MD5 Hash value: f5dc0d09f353095654de861af1a5c85d
Decimal value: 326803428735865603363252663499023042717
Random Number: 42969510585271152976449128897269169622091303081790275097780533455443251709748328780118525277430493361218528508294744637624121497408547775239658074
6544924566990282664819303780592678337429337819598525271427933261368824746250346128442762794888279530740219149001403177225081067217660269219213143528569561100878
Cipher text: 423813978726004088560293303043499112856115596935521644763626688117613133057229524348803945612529612017874722124890679761478888528243985473267262848
66651141440243845004910849502670336048202304246605890882175392532052315933186871711469298358109399238932509615852385538938619452281122359682180215011523302876836
734169390513549267032386104039221586817808526688174786303734687408995238639411171548750346474063658615139070331355157839012194030326458706378759022581552633674783
601031726934164422940242690857987896642156793956393957460895763843516289003358151035196467758118327265596325015368154128302883049311567073662136
Month 10: Income = 217
S412985711
MD5 Hash value: e959b9c055d22b44cf5d5ef8383b5
Decimal value: 310176004922442767264072821410178284309
Random Number: 43645432009111837191860633198397097700554393211146077038514036858013997263160193222828284467974426544954745562477569127253129301260213060017228144
099620574692641191507658022018917622234503932665706515277048716793521262961907095876823293120823231633632978164994352667142214569192829201982835525994461298135627
Cipher text: 2080236909575262947026526047332935613903864119296103489753809836089582859032337058236933128403323121450374830063714472769132327617358980702254443433
3703930876327856592685975742507758459065848810253881789847013284053873486439937029523026103053776619062482917266441478442441249859920357823554155826567390699788
166850244232533129045363526820688331944467872696364892213934198242044210162321207793223262874424386815114725166272508232750212162235510450740697500926094856678
04346430640349666348017710649450130107434094878162154339891107525760937155959546837182440055138360473282717723437083272604314332826546667650646
Month 11: Income = 4309
S412985712
MD5 Hash value: 587548a6ce880f2c2486ffa1344bd689
Decimal value: 117581035910806009520674382218617673353
Random Number: 87016497388102107682623991808204832583267549684734607991698021540969141451485513603008362782034757168112944561192944060979759194428852004864722853
3096829610215058353633719407474253635269299145461526405788704730590048706877064330543140311160839133667808987232461893678553741412630481510195519074122896406103
Cipher text: 588574949172297694234631154345446352603004065520064132454569944929589746796459545546158728160528609854303593055488069561192304592937743264408962905
6949623094689315914201232220911787526322340255063994639291382407711782381710213338220822734392560216470270228623757703728126325449734345515399691668235129659454
3574454221380756292771954466931651188513642099694450395367237029930808915864212151413327657445685620999175593016452443000407047714313880590858327295783461045324
774336018455297615544986830256250834305555925554826199974695499189839317879562316456215974241814234448545237977171656340945539612561159079254550
Month 12: Income = 3353

```

(3) How do you compute your average monthly salary in 2025 with the help of the cloud server (assume that the cloud server returns only one encrypted result to you)? What is the ciphertext computed by the cloud server?

The cloud server utilizes the **Additive Homomorphic Property** of the Paillier cryptosystem. Multiplying two or more Paillier ciphertexts together results in an encryption of the sum of their plaintext values.

$$c_1 \cdot c_2 \equiv g^{m_1+m_2} \cdot (r_1 \cdot r_2)^N \pmod{N^2}$$

To evaluate the data for the average, the cloud server cannot easily divide by 12 in the encrypted domain, so it simply computes the encrypted sum of all 12 months and returns this single ciphertext to the client.

The server calculates the product of all 12 ciphertexts modulo N^2 :

$$C_{sum} = \prod_{i=1}^{12} c_i \pmod{N^2}$$

The server returns C_{sum} and client has to decrypt that ciphertext using the private key before dividing by 12 to get the average monthly salary. We do it in next question.

$C_{sum} =$
4763902860973342825149433856904159841505756737971349989679595266205828031195371
2538846648889255909486124145200669540353185372411571576294270101718288001434054
7960062127286353245281789606002185234904363870282259079824230855760002557716761
9411551806911288849579248338702092007948740182261977237094050139607666626133935
0692854101900850428050199811920719153035173969058579962618060541495737211279522
7749155168812112889521454691334483879381251898490507729580139850443381118021339

9231203526554978485296958788582576244272020934825221288605647238978393847078337
571871376230847256170378054975758450781694741514792493718522773

```
Cloud returns Encrypted Sum: 47639028609733428251494338569041598415057567379713499896795952662058280311953
7125388466488892559094861241452006695403531853724115715762942701017182880014340547960062127286353245281789
6060021852349043638702822590798242308557600025577167619411551806911288849579248338702092007948740182261977
2370940501396076666261339350692854101900850428050199811920719153035173969058579962618060541495737211279522
7749155168812112889521454691334483879381251898490507729580139850443381118021339923120352655497848529695878
8582576244272020934825221288605647238978393847078337571871376230847256170378054975758450781694741514792493
718522773
```

(4) How do you decrypt the encrypted result from the cloud server to get your average monthly salary in 2025? Show the steps in detail.

Steps:

1. The client receives C_{sum} from the cloud. (The result as in the above question)
2. Apply Decryption Formula using the private key (p, q) to decrypt the sum. We have to use the decryption function for that,

$$m_{sum} = \left(\frac{C_{sum}^{(p-1)(q-1)} \pmod{N^2} - 1}{N} \right) \cdot [(p-1)(q-1)]^{-1} \pmod{N}$$

$m_{sum} = 68278$

3. Compute Average by dividing plaintext income m_{sum} by 12,

Average Monthly salary = 5689.83

```
Decrypted Total Income: 68278
Average Monthly Salary: 5689.833333333333
```

(5) Implement Paillier encryption algorithm (submit your code) and discuss the security of Paillier encryption algorithm.

Implemented in file `q1_homomorphic_encryption.py`

The security of the Paillier encryption algorithm relies on the *Decisional Composite Residuosity Assumption (DCRA)*. This mathematical assumption states that given a composite integer N and an integer z , it is computationally intractable to decide whether z is an N -residue modulo N^2 (That means whether there exists an y such that $z \equiv y^N \pmod{N^2}$).

Because of this property, Paillier achieves semantic security against chosen-plaintext attacks (IND-CPA). Furthermore, its additive homomorphic property makes it highly secure for cloud computing, as data administrators cannot access the plain data stored in the cloud, preventing data privacy breaches.

Q2. Key Recovery with Secret Sharing

(1) Select a suitable polynomial (using your student ID as one of the coefficient and determining what Paillier key parameter in Question 1 should be considered as the secret);

In the Paillier cryptosystem, the public key is (N, g) and the private key consists of the prime factors (p, q) . Because $N = p \times q$ is publicly known, knowing just *one* of the prime factors (Ex: p) is completely sufficient to calculate $q = N/p$ and reconstruct the entire private key.

Therefore, the secret S to be shared is 512-bit prime parameter p from question 1.

Defining the Polynomial,

According to Shamir's Secret Sharing scheme, to recover a secret with a minimum of t parts, you need a polynomial of degree $t-1$.

- Total shares (n): 10
- Threshold (t): 4
- Polynomial degree: $t - 1 = 3$

The general form of the polynomial is: $f(x) \equiv S + a_1x + a_2x^2 + a_3x^3 \pmod{P}$

- S is the secret (Paillier prime p).

$S =$

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466183

- P must be a prime number strictly larger than S .

$P =$

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466221

- a_1 is my student ID (4129857), as requested.
- a_2 and a_3 are randomly chosen integers where $0 < a < P$.

$a_2 = 987654321$

$a_3 = 192837465$

Then the polynomial,

$$f(x) \equiv S + 4129857x + 987654321x^2 + 192837465x^3 \pmod{P}$$

S and P values need to substitute from above values.

(2) Divide your Paillier key parameter into 10 shares (assume uploading them onto 10 different clouds, respectively);

To distribute the keys to the 10 clouds, we must calculate the y-value for each x-coordinate from 1 to 10 modulo P.

$$f(x) \equiv S + 4129857x + 987654321x^2 + 192837465x^3 \pmod{P}$$

We substitute 1, 2, 3, .. until 10 for x value get the output f(x) for each which are 10 shares.

```
--- Generating 10 Shares ---  
Cloud 1: x=1, y=1184621605  
Cloud 2: x=2, y=5501576680  
Cloud 3: x=3, y=14107889977  
Cloud 4: x=4, y=28160586286  
Cloud 5: x=5, y=48816690397  
Cloud 6: x=6, y=77233227100  
Cloud 7: x=7, y=114567221185  
Cloud 8: x=8, y=161975697442  
Cloud 9: x=9, y=220615680661  
Cloud 10: x=10, y=291644195632
```

(3) Recover your Paillier key parameter with any 4 of 10 shares (show the steps in detail);

Assume we retrieve the shares from Cloud 1, 2, 3, and 4. Our points are

$(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)$. According to the theory, I use the Lagrange basis polynomials to find $f(0)$,

$$f(0) = \sum_{i=1}^t y_i \prod_{i \neq j} \frac{x_j}{x_j - x_i} \pmod{P}$$

$$(x_1, y_1) = (1, 1184621605)$$

$$(x_2, y_2) = (2, 5501576680)$$

$$(x_3, y_3) = (3, 14107889977)$$

$$(x_4, y_4) = (4, 28160586286)$$

Step-by-step Expansion,

$$f(0) = y_1 \frac{x_2 x_3 x_4}{(x_2 - x_1)(x_3 - x_1)(x_4 - x_1)} + y_2 \frac{x_1 x_3 x_4}{(x_1 - x_2)(x_3 - x_2)(x_4 - x_2)} \\ + y_3 \frac{x_1 x_2 x_4}{(x_1 - x_3)(x_2 - x_3)(x_4 - x_3)} + y_4 \frac{x_1 x_2 x_3}{(x_1 - x_4)(x_2 - x_4)(x_3 - x_4)} \pmod{P}$$

$$f(0) = y_1 \frac{2 \cdot 3 \cdot 4}{(2-1)(3-1)(4-1)} + y_2 \frac{1 \cdot 3 \cdot 4}{(1-2)(3-2)(4-2)} + y_3 \frac{1 \cdot 2 \cdot 4}{(1-3)(2-3)(4-3)} \\ + y_4 \frac{1 \cdot 2 \cdot 3}{(1-4)(2-4)(3-4)} \pmod{P}$$

In modular arithmetic, we cannot perform standard division. To compute a fraction like,

$\frac{\text{numerator}}{\text{denominator}} \pmod{P}$ we must calculate the modular multiplicative inverse of the denominator modulo P , and then multiply it by the numerator.

First, I calculate the numerators and denominators for each of the four terms,

Term 1 Denominator: $(1)(2)(3) = 6$

Term 2 Denominator: $(-1)(1)(2) = -2 \pmod{P} = P - 2 =$

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466219

Term 3 Denominator: $(-2)(-1)(1) = 2$

Term 4 Denominator: $(-3)(-2)(-1) = -6 \pmod{P} = P - 6 =$

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466215

Next, using the Extended Euclidean Algorithm (implemented in my provided JavaScript code), I calculated the modular inverses for these denominators modulo P

Inverse of 6 $\pmod{P} =$

1007800624148392696767758455757769652929258889556927549090064760854754634847706
1771162347787410775949602683826520064762672995109021635980938290661976221851

Inverse of Term 2 =

6046803744890356180606550734546617917575553337341565294540388565128527809086237
062697408672446465569761610295912038857603797065412981588562974397185733110

Inverse of 2 $\pmod{P} =$

6046803744890356180606550734546617917575553337341565294540388565128527809086237
062697408672446465569761610295912038857603797065412981588562974397185733111

Inverse of Term 4 =

2015601248296785393535516911515539305858517779113855098180129521709509269695412
354232469557482155189920536765304012952534599021804327196187658132395244370

Substituting the actual y values, the numerators, and the modular inverses into the equation,

$$f(0) = (\text{Term 1}) + (\text{Term 2}) + (\text{Term 3}) + (\text{Term 4}) \pmod{P}$$

Term 1 = 4738486420

Term 2 =

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948761362006141

Term 3 = 56431559908

Term 4 =

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948766210879935

Final Recovered Secret (S),

S =

1209360748978071236121310146909323583515110667468313058908077713025705561817247
4125394817344892931139523220591824077715207594130825963177125948794371466183

This matches the original prime parameter p from Question 1, proving the secret was successfully recovered.

(4) Implement (1)-(3) with JavaScript (submit your codes);

Implemented in the file *q2_shamir's_secret.js*

In this we use BigInt because JavaScript's standard Number type loses precision with large integers for cryptographic calculations.

```
Term Denominator: 6
Modular Inverse of Denominator: 1007800624148392696767758455757769652929258889556927549090064760854754634847706177116234778741077594960268382652006476267299510902163
5980938290661976221851
Final Term Value: 4738486420
-----
Term Denominator: 120936074897807123612131014690932358351511066746831305890807771302570556181724741253948173448929311395232205918240777152075941308259631771259487943
71466219
Modular Inverse of Denominator: 604680374489035618060655073454661791757553337341565294540388565128527809086237062697408672446465569761610295912038857603797065412981
588562974397185733110
Final Term Value: 120936074897807123612131014690932358351511066746831305890807771302570556181724741253948173448929311395232205918240777152075941308259631771259487613
62006141
-----
Term Denominator: 2
Modular Inverse of Denominator: 604680374489035618060655073454661791757553337341565294540388565128527809086237062697408672446465569761610295912038857603797065412981
588562974397185733111
Final Term Value: 56431559908
-----
Term Denominator: 120936074897807123612131014690932358351511066746831305890807771302570556181724741253948173448929311395232205918240777152075941308259631771259487943
71466215
Modular Inverse of Denominator: 2015601248296785393535516911515539305858517779113855098180129521709509269695412354232469557482155189920536765304012952534599021804327
196187658132395244370
Final Term Value: 120936074897807123612131014690932358351511066746831305890807771302570556181724741253948173448929311395232205918240777152075941308259631771259487662
10879935
-----
--- Recovering Secret ---
Recovered Secret: 120936074897807123612131014690932358351511066746831305890807771302570556181724741253948173448929311395232205918240777152075941308259631771259487943
71466183
Matches Original: true
```

(5) Analyse the security of your Paillier key parameter in the case of 3 of the 10 clouds collude to derive your decryption key.

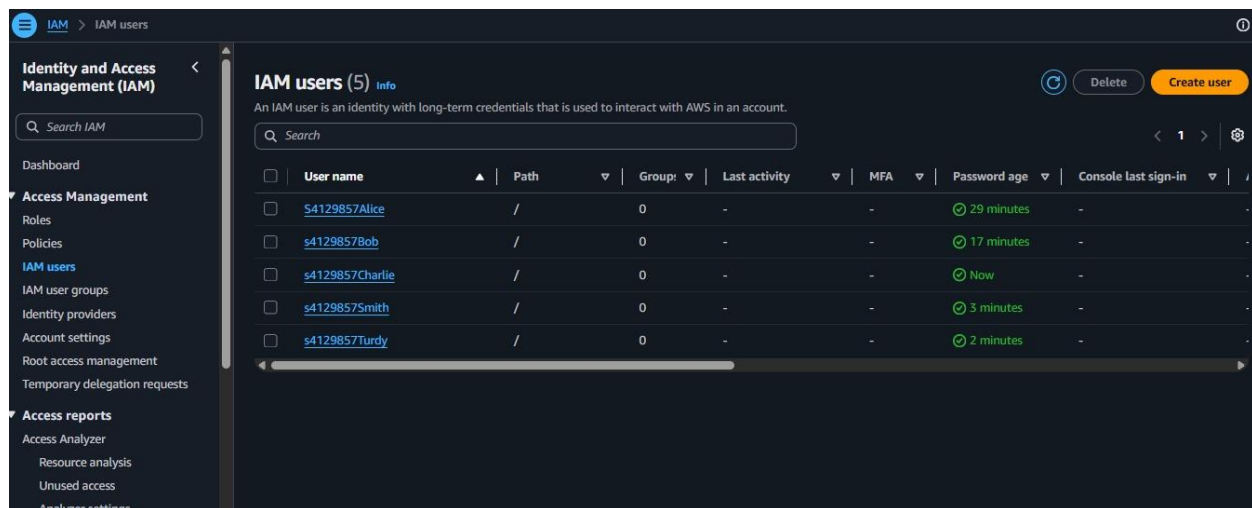
If 3 out of the 10 clouds collude to derive the decryption key, it will fail completely.

Shamir's Secret Sharing scheme is known to possess **perfect secrecy**. Because we used a $(t, n) = (4, 10)$ threshold scheme, a minimum of 4 distinct shares is mathematically required to construct the degree-3 polynomial.

If exactly 3 clouds collude, they only possess 3 points on the polynomial graph. Through any 3 points, there exist an infinite number of possible degree-3 curves that can be drawn. Therefore, for every single possible value of the secret S (from 0 to $P-1$), there is exactly one unique polynomial that fits the 3 colluding clouds' points and intersects the y-axis at that assumed S . Because every single possible secret remains equally likely, the colluding clouds gain absolutely zero partial information about the true Paillier decryption key.

Q3. Secure Data Management via Amazon S3

3.1. In order for the five staff to upload and download data to and from Amazon S3, create user accounts for them (please name the user with your student ID + Alice, Bob, Smith, Turdy, Charlie, e.g., s1234567Alice).



3.2. In order to protect data privacy between departments, different department buckets should be encrypted by different secret keys. Generate the marketing key, the sales key, and the services key, respectively. Allow Alice and Bob to access the marketing key, Smith and Trudy to access the sales key, and Charlie to access the services key.

The screenshot shows the AWS IAM console with the 'IAM users' page selected. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, and Access reports. The main content area displays a table of IAM users with the following columns: User name, Path, Group, Last activity, MFA, Password age, and Console last sign-in. There are 5 users listed.

User name	Path	Group	Last activity	MFA	Password age	Console last sign-in
s4129857Alice	/	0	-	-	29 minutes	-
s4129857Bob	/	0	-	-	17 minutes	-
s4129857Charlie	/	0	-	-	Now	-
s4129857Smith	/	0	-	-	3 minutes	-
s4129857Turdy	/	0	-	-	2 minutes	-

This screenshot is identical to the one above, showing the AWS IAM console with the 'IAM users' page. It displays a table of 5 IAM users with their respective details.

User name	Path	Group	Last activity	MFA	Password age	Console last sign-in
s4129857Alice	/	0	-	-	29 minutes	-
s4129857Bob	/	0	-	-	17 minutes	-
s4129857Charlie	/	0	-	-	Now	-
s4129857Smith	/	0	-	-	3 minutes	-
s4129857Turdy	/	0	-	-	2 minutes	-

3.3 Create three buckets in S3 for the marketing, sales and services departments, respectively. Encrypt the marketing, sales and services department buckets with the marketing key, the sales key, and the services key, respectively.

The screenshot shows the AWS S3 console with the 'Buckets' page selected. The left sidebar contains navigation links for Amazon S3, Buckets, Files, Access management and security, and Storage management and insights. The main content area displays the configuration for a bucket named 's4129857-services'. The configuration includes Default encryption, Encryption type, Encryption key ARN, Bucket Key, and Intelligent-Tiering Archive configurations.

Default encryption
Server-side encryption is automatically applied to new objects stored in this bucket.

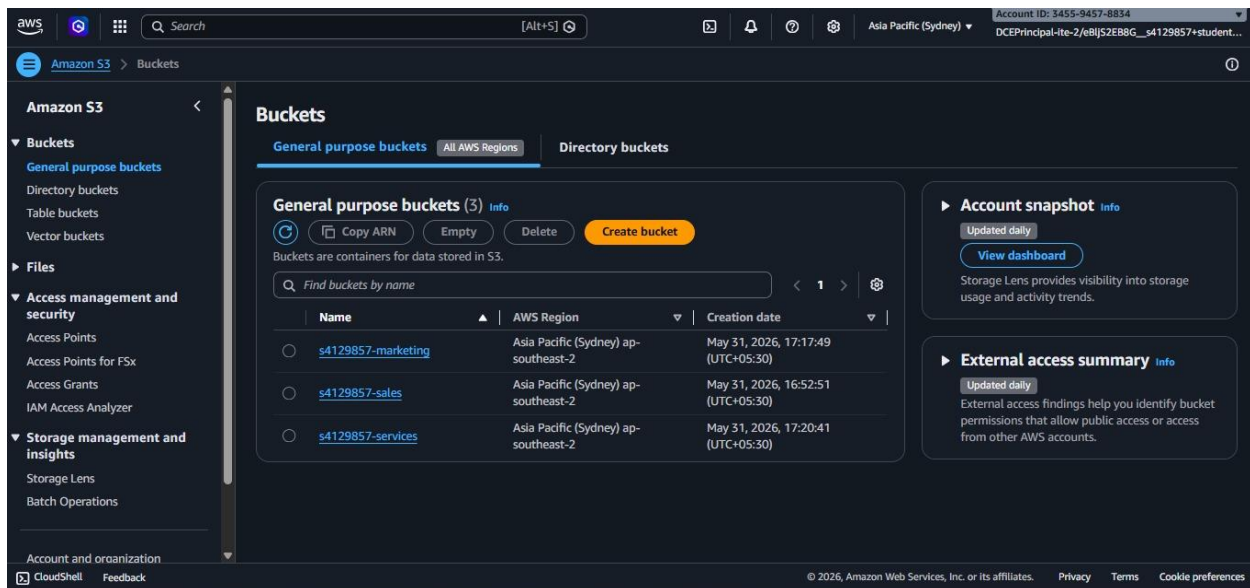
Encryption type
Server-side encryption with AWS Key Management Service keys (SSE-KMS)

Encryption key ARN
[arn:aws:kms:ap-southeast-2:345594578834:key/b369e0eb-58a9-4d45-90cd-c883ab459ce9](#)

Bucket Key
When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS. [Learn more](#)
Enabled

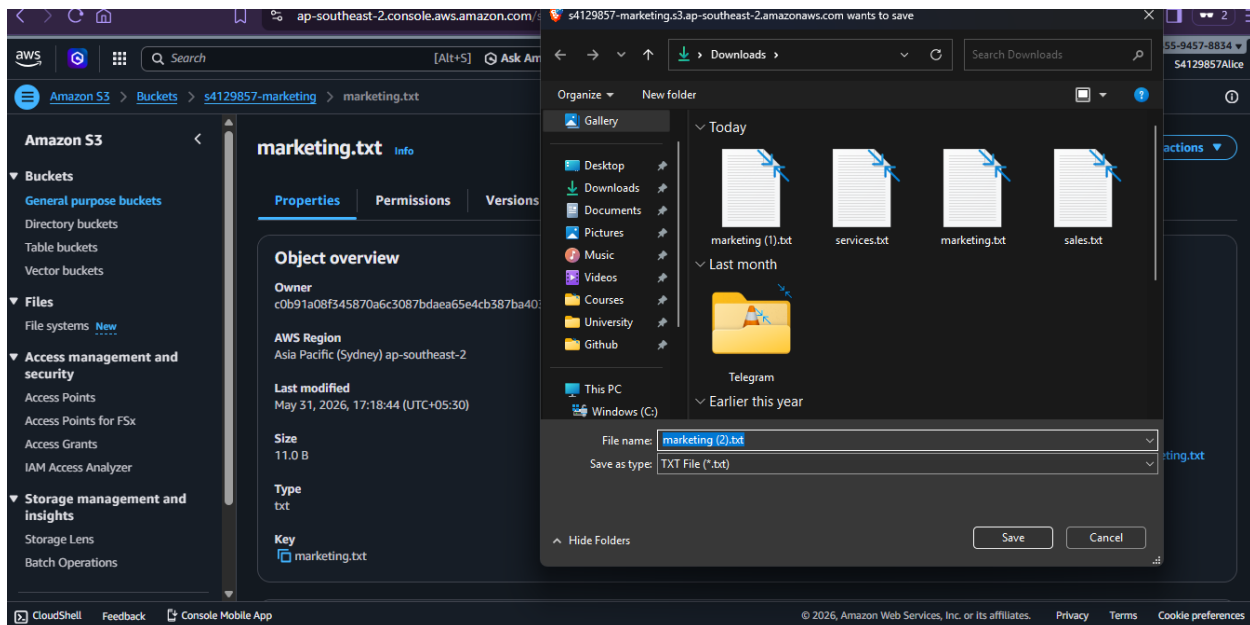
Blocked encryption types
Server-Side Encryption with Customer-Provided Keys (SSE-C)

Intelligent-Tiering Archive configurations (0)
Enable objects stored in the Intelligent-Tiering storage class to tier-down to the Archive Access tier or the Deep Archive Access tier which are optimized for objects that will be rarely accessed for long periods of time. [Learn more](#)



3.4 Demonstrate how Alice is able to get access to an object of the bucket for the marketing department and how Alice is unable to get access to an object of the bucket for either the sales or the services department with examples.

From alice account he only have permission to Marketing s3 bucket objects only.



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>User: arn:aws:iam::345594578834:user/54129857Alice is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-southeast-2:345594578834:key/58503bd5-5693-42c1-9181-a83c56ff2755 because no identity-based policy allows the kms:Decrypt action</Message>
  <RequestId>CT2YH6NZYABW84Q</RequestId>
  <HostId>mYK1vsLg754yE1gIt+8rHSDASwWb0ap2muAh0v1YIYAfGeell70b4IUIX2JfzGwYUj3iI+ttU4w7nCkCCY6jv/Tl3M49uW</HostId>
</Error>
```

Permission is denied from other two buckets (sales, services).

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>User: arn:aws:iam::345594578834:user/54129857Alice is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:ap-southeast-2:345594578834:key/b369e0eb-58a9-4d45-90cd-c883ab459ce9 because no identity-based policy allows the kms:Decrypt action</Message>
  <RequestId>0K2K3NRXY57FC32K</RequestId>
  <HostId>u8xR/3rB4MoeyFPLG3i1nF7IqvGeaIX/bMg2u/V8+tm1XGNeStcJwJ8yL2htSkd306/+eBnDPHI=</HostId>
</Error>
```

Q4. Create AWS Client Virtual Private Network

4.1 Create an AWS VPN Server in AWS and configure the AWS VPN Server connection.

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name
vpn-server [Add additional tags](#)

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

[AMI from catalog](#) [Quick Start](#)

Name
OpenVPN Access Server Community Image-fe8020db-5343-4c43-9e65-5ed4a825c931 Verified provider

Description
OpenVPN Access Server 3.1.0 publisher image from <https://www.openvpn.net/>.

Image ID
ami-0af8e99f388cc48e6

Username ⓘ
root (Check with the AMI provider.)

Summary

Number of instances Info
1

Software Image (AMI)
OpenVPN Access Server / Self-H...[read more](#)
ami-0af8e99f388cc48e6

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet. Data transfer charges are not included.

AMI from catalog [Quick Start](#)

Name
OpenVPN Access Server Community Image-fe8020db-5343-4c43-9e65-5ed4a825c931 Verified provider

Description
OpenVPN Access Server 3.1.0 publisher image from <https://www.openvpn.net/>.

Image ID
ami-0af8e99f388cc48e6

Username ⓘ
root (Check with the AMI provider.)

Catalog	Published	Architecture	Virtualization	Root device type	ENA Enabled
AWS Marketplace AMIs	2026-03-17T11:04:38.000Z	x86_64	hvm	ebs	Yes

Boot mode
uefi-preferred

If you have an existing license entitlement to use this software, then you can launch this software without creating a new subscription. If you do not have an existing entitlement, then by launching this software, you will be subscribed to this software and agree that your use of this software is subject to the pricing terms and the seller's [End User License Agreement](#).

Summary

Number of instances Info
1

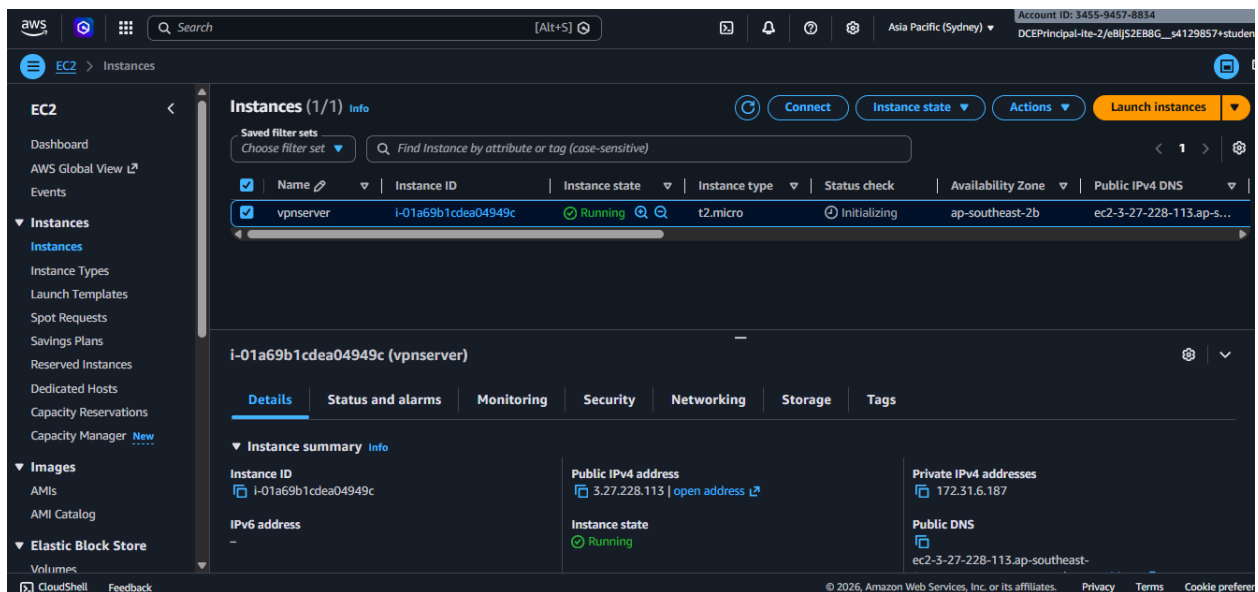
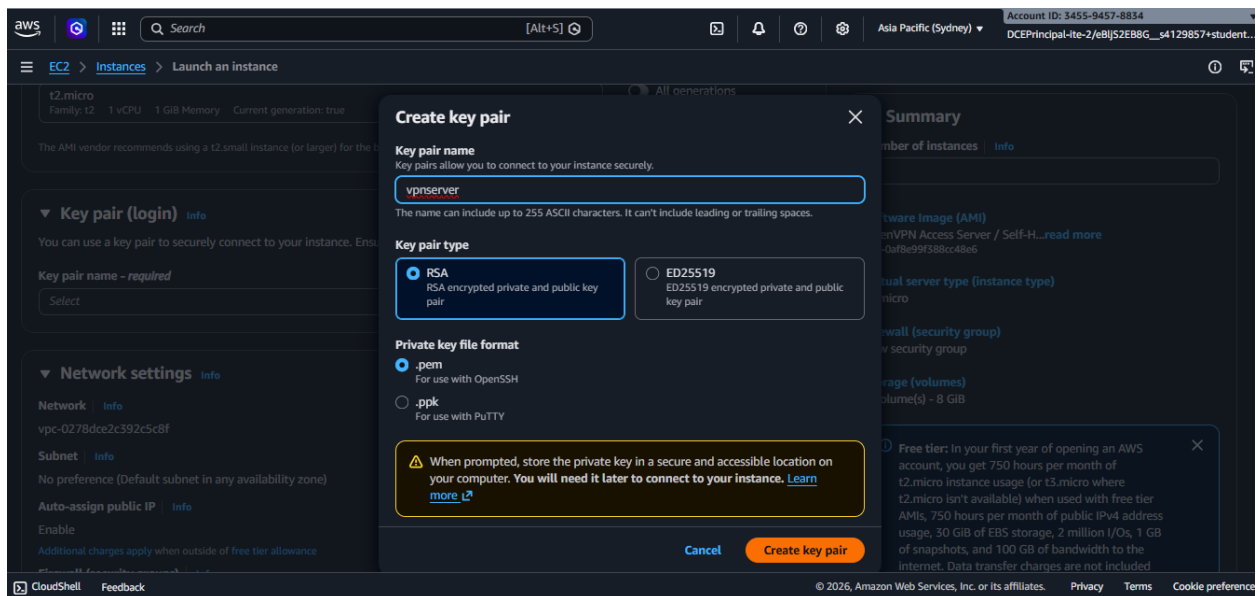
Software Image (AMI)
OpenVPN Access Server / Self-H...[read more](#)
ami-0af8e99f388cc48e6

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

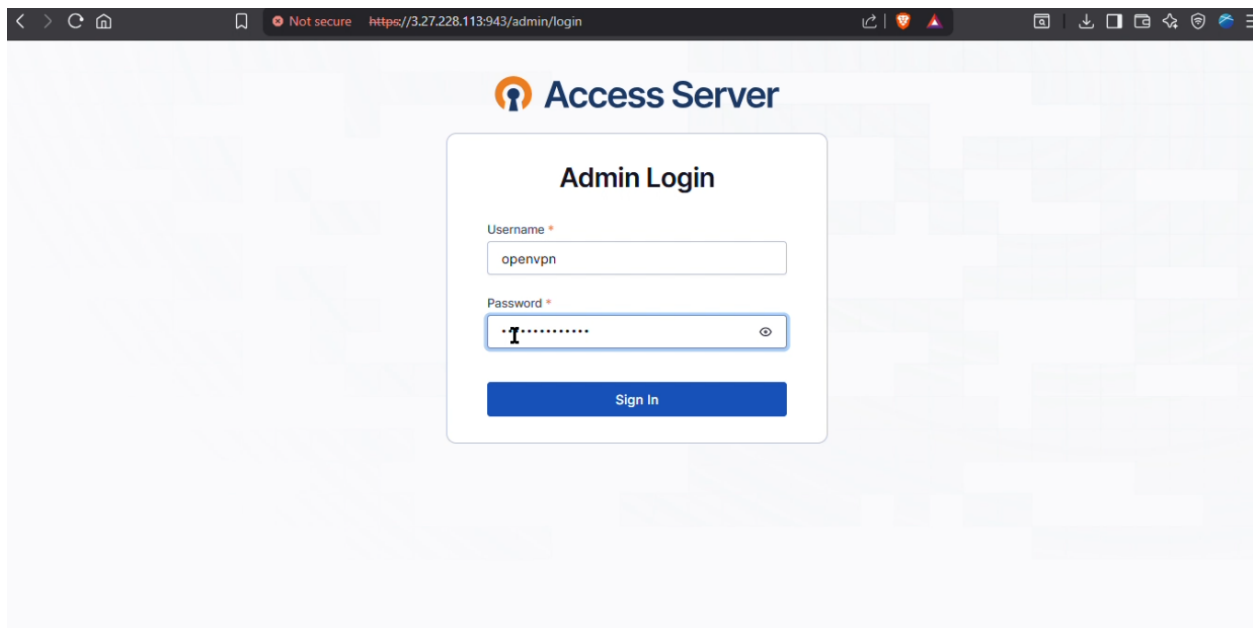
Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet. Data transfer charges are not included.



```
openvpnas@openvpnas3: ~  
C:\Users\thath\Downloads>ssh -i "vpnsrvr.pem" openvpnas@ec2-3-27-228-113.ap-southeast-2.compute.amazonaws.com  
The authenticity of host 'ec2-3-27-228-113.ap-southeast-2.compute.amazonaws.com (3.27.228.113)' can't be established.  
ED25519 key fingerprint is SHA256:UE1lBpZVYfZfvM3TWLeNCVqts2G1r8KcUm8NL1DTqs4.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-3-27-228-113.ap-southeast-2.compute.amazonaws.com' (ED25519) to the list of known hosts.  
  
Welcome to OpenVPN Access Server Appliance 3.1.0  
  
* Documentation: https://help.ubuntu.com  
* Management: https://landscape.canonical.com  
* Support: https://ubuntu.com/pro  
  
System information as of Sun May 31 12:28:16 UTC 2026  
  
System load: 0.0          Processes:          107  
Usage of /: 36.6% of 6.71GB Users logged in:    0  
Memory usage: 23%        IPv4 address for enX0: 172.31.6.187  
Swap usage: 0%  
  
* Ubuntu Pro delivers the most comprehensive open source security and  
  compliance features.  
  
  https://ubuntu.com/aws/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.
```

4.2 Setup a password for your staff to get access to the AWS VPN Server. The password should be your student ID.

```
Initial version is not set. Setting it to 3.1.0...  
Generating PAM config for openvpnas ...  
Enabling service  
Created symlink /etc/systemd/system/multi-user.target.wants/openvpnas.service → /usr/lib/systemd/system/openvpnas.service.  
Starting openvpnas...  
  
NOTE: Your system clock must be correct for OpenVPN Access Server  
to perform correctly. Please ensure that your time and date  
are correct on this system.  
  
Initial Configuration Complete!  
  
You can now continue configuring OpenVPN Access Server by  
directing your Web browser to this URL:  
  
https://3.27.228.113:943/admin  
  
During normal operation, OpenVPN AS can be accessed via these URLs:  
Admin UI: https://3.27.228.113:943/admin  
Client UI: https://3.27.228.113:943/  
To login please use the "openvpn" account with the password you specified during the setup.  
  
See the Release Notes for this release at:  
  https://openvpn.net/vpn-server-resources/release-notes/  
  
openvpnas@openvpnas3:~$
```



Access Server3.1.0

Status

VPN Server

Users

Groups

Access Controls

Authentication

Web Services

openvpn

Post-Auth Plugins

Documentation

Support

Logout

0openvpn

Status

Overview

Active Connections

Activity Log

Connections

View active connections

0

1

2

In use (on this node)

0

Available

2

Total

2

License

Activate License

License typeUnlicensed (2 connections)

Current Activation stateActive

Server Details

Server address3.27.228.113

Access Server version3.1.0

Operating SystemUbuntu 24.04.4 LTS

OS hostnameopenvpnas3

Accepting connections onall

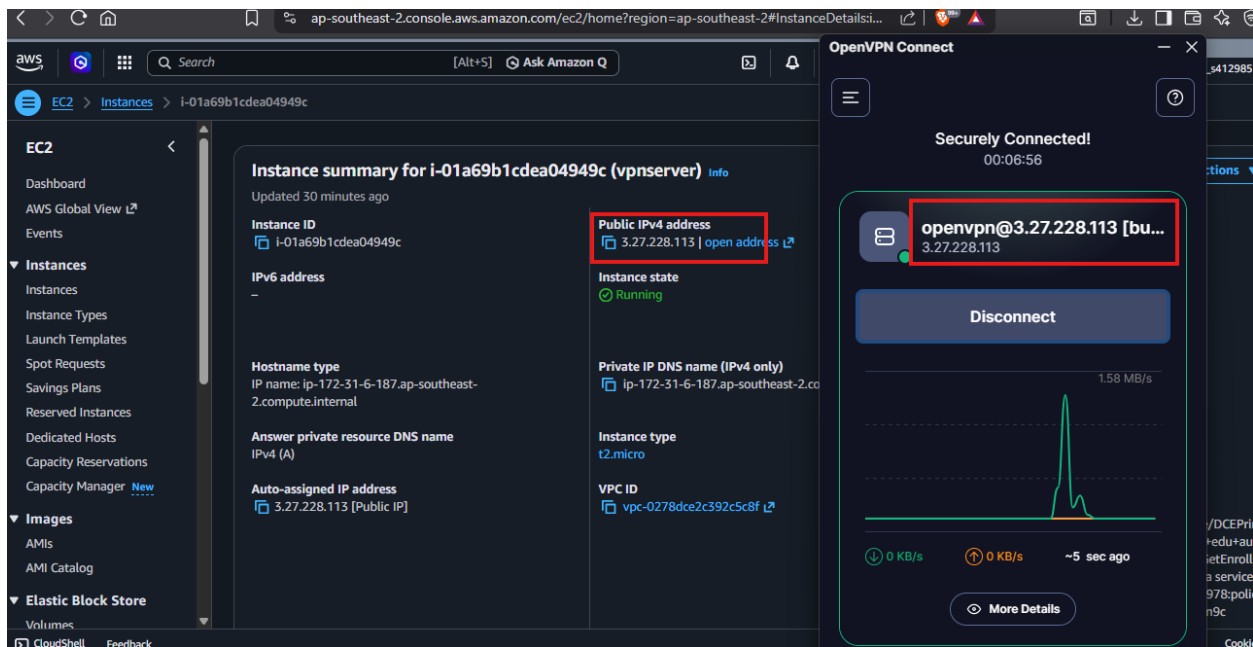
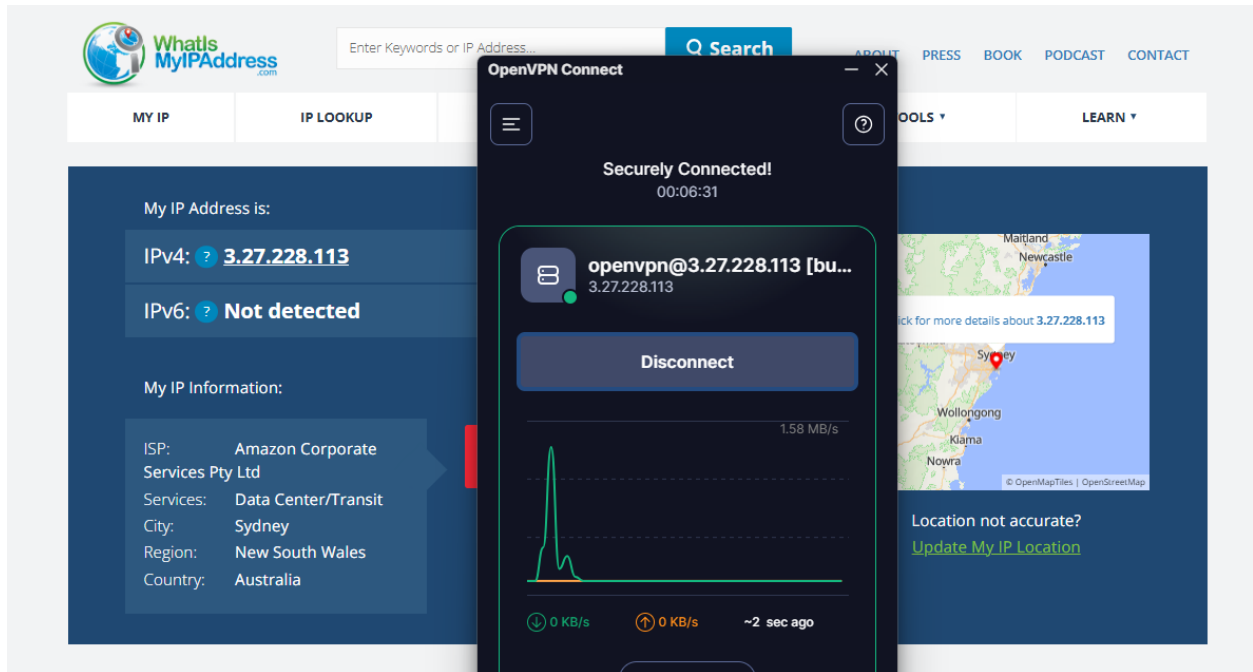
Porttcp/443, udp/1194

Default authentication systemlocal

Accepting authentication onlocal, pam

Running with version OpenVPN 2.6.16as1 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [MH/PKTINFO] [AEAD] [DCO]

4.3. Demonstrate admin login and client login to the AWS VPN Server.



4.4 Jane, the CEO of Bookshop PTY LTD is concerned about the security of the VPN solution. Explain to Jane how secure your solution is.

To ensure the utmost protection for ABC Bookshop's transition to the cloud, the implemented AWS VPN Server utilizes industry-standard security protocols to protect both the company's data and its infrastructure. The security of this solution rests on three core pillars:

1. Data Confidentiality and Encryption (SSL/TLS)- When staff members connect to the VPN from remote locations, the OpenVPN software establishes a secure, encrypted tunnel over the public internet. This means that all sensitive company data, login credentials, and internal communications are converted into unreadable ciphertext before leaving the staff member's

computer. Because of this strong encryption, even if a cybercriminal intercepts the traffic on a public Wi-Fi network, they cannot read or steal the data.

2. Network Isolation (VPC Architecture)- By implementing this VPN, we do not have to expose ABC Bookshop's sensitive databases or internal web servers to the public internet. Instead, these resources are safely isolated inside an Amazon Virtual Private Cloud (VPC). The VPN acts as a heavily guarded front door. Only after a staff member successfully authenticates at the Client VPN endpoint are they granted a secure pathway into the private subnets (e.g., 172.31.0.0/20). To the outside world, the internal network remains completely invisible.

3. Data Integrity and Threat Mitigation- The underlying cryptographic protocols (like SSL/TLS) continuously verify message and connection integrity. This prevents Man-in-the-Middle (MITM) attacks, ensuring that malicious actors cannot secretly alter financial records or database commands while they are in transit. Furthermore, strict authentication requires a specific username and password combination, ensuring only verified employees can access the VPN server in the first place.

This VPN solution ensures that ABC Bookshop's data remains confidential, its internal cloud infrastructure remains hidden from public view, and all remote staff communications are protected against interception and tampering.